ĐẶC TẢ HỆ THỐNG · PHIÊN BẢN 1.0

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

Systolic array Output-Stationary · DMA · Khối hậu xử lý vector (VPU)

Đề tài: Thiết kế bộ tăng tốc AI tích hợp trong vi xử lý RISC-V

Phiên bản	1.0 — 05/11/2026
Nội dung	Kiến trúc, chức năng, mô hình lập trình, luồng vận hành và port top-level của `gemm_accel_top`
Port top-level	**Đã khóa.** Thay đổi về sau chỉ xảy ra ở port nội bộ, được mô tả trong spec của từng module
Phần bổ sung	DMA theo chuỗi descriptor và scratchpad ping-pong (mục 7): làm nếu còn thời gian, không làm thay đổi port top-level hay các thanh ghi đã có

Mục lục

1 Tổng quan **3**
1.1 Phạm vi chức năng . . . 3
1.2 Quy ước . . . 3
1.3 Sơ đồ khối . . . 3
1.4 Module . . . 4
1.5 Tham số hệ thống . . . 4

2 Chức năng và dữ liệu **4**
2.1 Phép toán . . . 4
2.2 Kích thước ma trận . . . 5
2.3 Bố trí dữ liệu trong DRAM . . . 5
2.4 Mô hình tham chiếu . . . 6

3 Mô hình lập trình **6**
3.1 Register map . . . 6
3.2 Yêu cầu thanh ghi . . . 7
3.3 Ngữ nghĩa tác vụ . . . 8
3.4 Quy tắc cho firmware . . . 8

4 Luồng vận hành **9**
4.1 Các pha của một tác vụ . . . 9
4.2 Luồng một tác vụ . . . 10
4.3 Quyền truy cập scratchpad theo pha . . . 11
4.4 Xử lý lỗi . . . 11
4.5 Reset và soft reset . . . 11

5 Vận hành của các khối **12**
5.1 Register File . . . 12
5.2 Main FSM . . . 13
5.3 DMA engine . . . 13
5.4 Local SRAM . . . 14
5.5 Systolic array . . . 14
5.6 VPU . . . 15

6 Port top-level **16**
6.1 Tham số . . . 16
6.2 Danh sách port . . . 16

7 Thành phần bổ sung **17**
7.1 Nguyên tắc chung . . . 18
7.2 DMA theo chuỗi descriptor . . . 18
7.3 Scratchpad ping-pong . . . 19

1 Tổng quan

`gemm_accel_top` là bộ tăng tốc nhân ma trận INT8 gắn vào Chipyard SoC. CPU chỉ ghi cấu hình qua MMIO và chờ kết quả. Accelerator tự dùng DMA chép A, B từ DRAM vào scratchpad, tính C = A × B trên systolic array, cho kết quả đi qua **VPU** để cộng bias, requantize và ReLU nếu được bật, rồi tự ghi kết quả về DRAM.

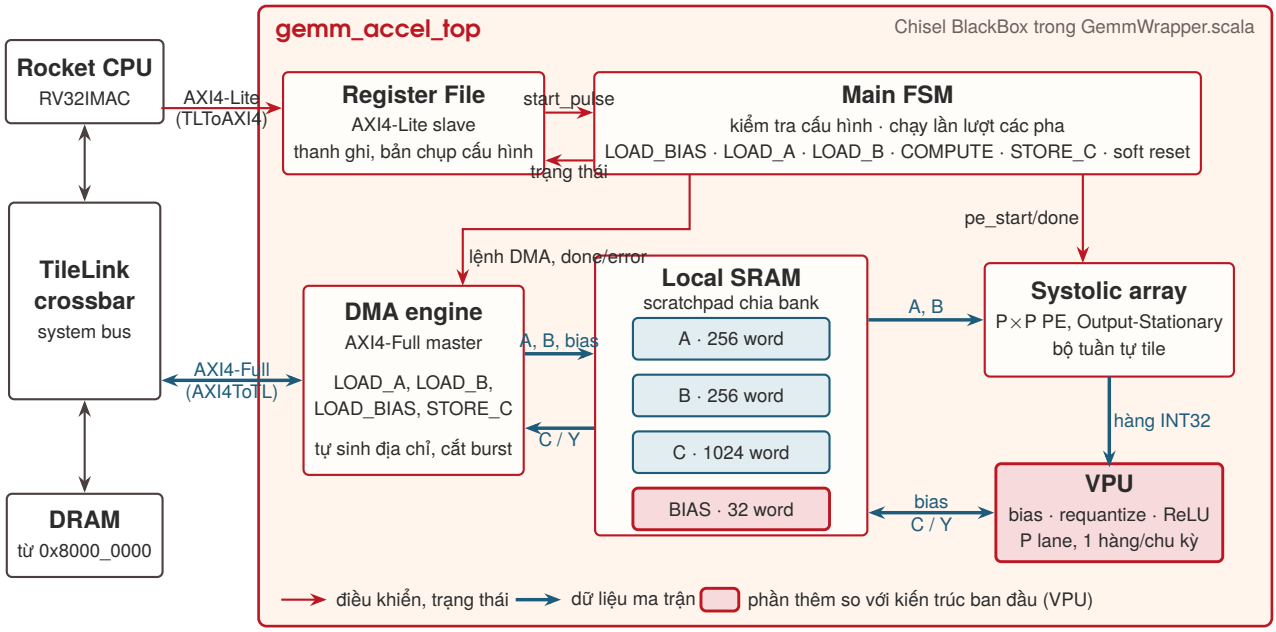
1.1 Phạm vi chức năng

- Mỗi tác vụ tính một phép GEMM: A là M×K, B là K×N, kiểu INT8 có dấu; tích lũy INT32; M ≤ 32, N ≤ 32, K ≤ 32.
- Kết quả ra DRAM ở một trong hai dạng: C (INT32, không hậu xử lý) hoặc Y (INT8, sau bias, requantize, ReLU tùy chọn, kẹp).
- CPU cấu hình qua AXI4-Lite (MMIO) và chờ kết quả bằng polling; DMA đọc/ghi DRAM qua AXI4-Full.
- Các pha của một tác vụ chạy tuần tự: nạp dữ liệu, tính, ghi kết quả.
- Không có: ngắt, C += A × B, ma trận lớn hơn 32 trong một tác vụ, dataflow Weight-Stationary, bộ nhớ ảo.

1.2 Quy ước

- "phải" = bắt buộc; "nên" = khuyến nghị; "được phép" = tùy chọn. Dòng có mã (ví dụ **REG-03**) là yêu cầu; hình vẽ và ví dụ chỉ để minh họa.
- M, N, K: A là M×K, B là K×N, C và Y là M×N. P: kích thước array vật lý (P×P PE). *word* = 32 bit; *beat* = một lần truyền AXI 32 bit; *tile* = khối P×P của C mà array tính trong một lượt.
- *Tác vụ*: từ khi lệnh START được chấp nhận đến khi STATUS báo DONE hoặc ERROR. *Bản chụp cấu hình*: bản sao các thanh ghi cấu hình, lấy đúng lúc START được chấp nhận và dùng suốt tác vụ.
- Màu trong hình: **đỏ** là điều khiển và trạng thái, **xanh** là dữ liệu ma trận.

1.3 Sơ đồ khối



Hình 1.1: Sơ đồ khối `gemm_accel_top`. Không vẽ: bộ đồng bộ reset; bản chụp cấu hình từ Register File tới FSM, DMA, Array, VPU; `rst_n_sync` và `soft_clr` tới mọi module

1.4 Module

Bảng 1.1: Module trong `gemm_accel_top`

Module	Instance	Vai trò
`gemm_register_file`	`u_regfile`	AXI4-Lite slave; thanh ghi cấu hình, trạng thái, bộ đếm; chấp nhận START; giữ bản chụp cấu hình
`gemm_main_fsm`	`u_main_fsm`	Kiểm tra cấu hình; chạy lần lượt các pha của tác vụ; ra lệnh cho DMA và array; báo DONE/ERROR; điều phối soft reset
`gemm_dma_engine`	`u_dma`	AXI4-Full master; chép DRAM ↔ scratchpad theo lệnh LOAD_A, LOAD_B, LOAD_BIAS, STORE_C
`gemm_local_sram`	`u_sram`	Buffer A, B, C, BIAS, chia bank bên trong để phục vụ array và VPU mỗi chu kỳ
`gemm_systolic_array`	`u_array`	Mảng P×P PE (`gemm_pe`), làm lệch dữ liệu, bộ tuần tự tile, xả kết quả theo hàng
`gemm_vpu`	`u_vpu`	Hậu xử lý trên đường xả kết quả: bias, requantize, ReLU, kẹp INT8; đi thẳng khi tắt

Ngoài 6 module trên, `gemm_accel_top` có một bộ đồng bộ reset 2 flip-flop (RST-01).

1.5 Tham số hệ thống

Bảng 1.2: Tham số hệ thống

Tham số	Giá trị	Ghi chú
`PE_ARRAY_SIZE` (P)	4 hoặc 8; mặc định 8	Kết quả không phụ thuộc P
`MAX_DIM`	32	Giới hạn M, N, K
`AXI_DATA_WIDTH`	32	Cả AXI4-Lite và AXI4-Full
`MAX_BURST_BEATS`	16 (mặc định)	Đặt theo giới hạn của bridge
`SRAM_RD_LAT`	1 chu kỳ	Cổng phía array và VPU
Buffer A / B / C / BIAS	256 / 256 / 1024 / 32 word	Tổng 6,125 KiB
`MMIO_BASE`	0x6000_0000, vùng 4 KiB	
`DRAM_BASE`	0x8000_0000	
Burst outstanding	1 đọc + 1 ghi	

2 Chức năng và dữ liệu

> Mỗi tác vụ tính C = A × B trên INT8, tích lũy INT32. Khi `POST_EN` = 0, C (INT32) được ghi ra DRAM. Khi `POST_EN` = 1, VPU biến C thành Y = kẹp(ReLU(requantize(C + bias))) ở INT8 và Y được ghi ra DRAM. Mọi phép tính là số nguyên và kết quả phải khớp từng bit với mô hình tham chiếu ở mục 2.4.

2.1 Phép toán

Bảng 2.1: Phép toán

ID	Yêu cầu
FUN-01	$C[i][j] = \sum_{k=0}^{K-1} A[i][k] \cdot B[k][j]$, với 0 ≤ i < M, 0 ≤ j < N
FUN-02	A, B là INT8 bù 2. C và bias là INT32 bù 2. Y là INT8 bù 2. Tích INT8 × INT8 tính đủ 16 bit có dấu; cộng dồn trên INT32, wrap mod 2^{32} nếu tràn

Bảng 2.1: Phép toán (tiếp theo)

ID	Yêu cầu
FUN-03	Khi POST_EN = 1, với mỗi phần tử: $acc = C[i][j] + bias[j]$ (INT32, wrap mod 2^{32}) $prod = acc \cdot M_0$ (tích có dấu 64 bit) $y = (prod + 2^{30+sh}) \gg (31 + sh)$ (dịch số học, 64 bit) nếu RELU_EN = 1: $y = \max(y, 0)$ $Y[i][j] = \min(\max(y, -128), 127)$
FUN-04	M_0 (QUANT_M0) là INT32 có dấu, mọi giá trị đều hợp lệ. sh (QUANT_SHIFT) là số không dấu 0 … 31. Một cặp (M_0, sh) dùng cho cả tác vụ. bias là vector N phần tử INT32, cộng theo cột
FUN-05	Kết quả ghi đè vùng đích trong DRAM; accelerator không đọc giá trị cũ của vùng đích
FUN-06	Ngoài FUN-01 và FUN-03 không có phép toán nào khác (không có C += A × B, softmax, pooling)
FUN-07	Kết quả không phụ thuộc P: P = 4 và P = 8 cho nội dung DRAM giống hệt từng bit

HỆ QUẢ
$|a \cdot b| \le 2^{14}$ và K ≤ 32 nên $|C| \le 2^{19}$: C không bao giờ tràn. $|prod| \le 2^{62}$ và $2^{30+sh} \le 2^{61}$ nên phép tính 64 bit của FUN-03 không bao giờ tràn. Làm tròn là nửa lên (về phía +∞). Phần mềm nên chọn M_0 trong $[2^{30}, 2^{31})$ để giữ độ chính xác.

2.2 Kích thước ma trận

Bảng 2.2: Bố cục MATRIX_DIM

Bit	Trường	Miền hợp lệ
[7:0]	M (số hàng A và C)	1 … 32
[15:8]	N (số cột B và C)	4, 8, …, 32 (bội của 4)
[23:16]	K (số cột A = số hàng B)	4, 8, …, 32 (bội của 4)
[31:24]	Dự trữ	phải = 0

Ma trận có N hoặc K không chia hết cho 4 phải được phần mềm đệm 0 lên bội của 4; cột thừa của kết quả bị phần mềm bỏ qua. Toàn bộ A, B, bias và C của một tác vụ nằm trong scratchpad suốt tác vụ.

2.3 Bố trí dữ liệu trong DRAM

Bảng 2.3: Địa chỉ byte của phần tử

Ma trận	Địa chỉ	Kích thước vùng
A	BASE_ADDR_IN + i·K + k	M·K byte
B	BASE_ADDR_WEIGHT + k·N + j	K·N byte
bias	BIAS_ADDR + 4j	4·N byte
C (INT32)	BASE_ADDR_OUT + 4·(i·N + j)	4·M·N byte
Y (INT8)	BASE_ADDR_OUT + i·N + j	M·N byte

Bảng 2.4: Bố trí dữ liệu

ID	Yêu cầu
DAT-01	Row-major, các hàng nằm liền nhau, không có stride, theo bảng 2.3
DAT-02	Little-endian: byte tại địa chỉ a nằm ở bit [8·(a mod 4)+7 : 8·(a mod 4)] của word 32 bit; INT32 là little-endian
DAT-03	Mọi địa chỉ cơ sở căn 4 byte. Vì N, K là bội của 4, mọi hàng bắt đầu và kết thúc đúng biên word, nên m_axi_wstrb luôn bằng 4'b1111

Bảng 2.4: Bố trí dữ liệu (tiếp theo)

ID	Yêu cầu
DAT-04	Vùng đích không được chồng lên vùng A, B hoặc bias; A, B, bias được phép chồng nhau. Vi phạm → kết quả không xác định. Phần cứng không kiểm tra
DAT-05	Phần cứng không kiểm tra vùng nhớ có nằm trong DRAM không; bus trả lỗi thì tác vụ kết thúc với ERROR (mục 4.4)

2.4 Mô hình tham chiếu

```
import numpy as np

def gemm(A, B):                        # A: int8 MxK, B: int8 KxN
    return (A.astype(np.int64) @ B.astype(np.int64)).astype(np.int32)

def post(C, bias, M0, sh, relu):       # C: int32 MxN, bias: int32 N
    acc  = (C.astype(np.int64) + bias).astype(np.int32).astype(np.int64)
    prod = acc * np.int64(M0)
    y    = (prod + (np.int64(1) << (30 + sh))) >> (31 + sh)
    if relu:
        y = np.maximum(y, 0)
    return np.clip(y, -128, 127).astype(np.int8)
```

3 Mô hình lập trình

Firmware ghi A, B (và bias) vào DRAM, ghi kích thước, địa chỉ và tham số hậu xử lý vào thanh ghi, ghi START, rồi đọc STATUS cho tới khi DONE hoặc ERROR bằng 1. Register map dưới đây đã khóa: phần bổ sung ở mục 7 chỉ dùng các offset và bit đang để dự trữ cho nó.

3.1 Register map

Địa chỉ tuyệt đối = `MMIO_BASE` + offset. Vùng MMIO rộng 4 KiB (0x6000_0000 … 0x6000_0FFF).

Bảng 3.1: Register map

Offset	Tên	Truy cập	Reset	Nội dung
0x00	`CTRL_REG`	W (đọc 0)	0	[0] START, [1] SOFT_RESET, [2] START_CHAIN (mục 7.2), [31:3] dự trữ
0x04	`STATUS_REG`	R	0	[0] DONE, [1] BUSY, [2] ERROR, [3] MODE, [7:4] `ERR_CODE`, [31:8] = 0
0x08	`MATRIX_DIM`	R/W	0	M, N, K (bảng 2.2)
0x0C	`BASE_ADDR_IN`	R/W	0	Địa chỉ byte của A
0x10	`BASE_ADDR_WEIGHT`	R/W	0	Địa chỉ byte của B
0x14	`BASE_ADDR_OUT`	R/W	0	Địa chỉ byte của C hoặc Y
0x18	`POST_CTRL`	R/W	0	[0] `POST_EN`, [1] `RELU_EN`, [31:2] dự trữ
0x1C	`BIAS_ADDR`	R/W	0	Địa chỉ byte của vector bias
0x20	`QUANT_M0`	R/W	0	M_0 của FUN-03
0x24	`QUANT_SHIFT`	R/W	0	[4:0] sh của FUN-03, [31:5] dự trữ
0x28	`HW_CONFIG`	R	hằng	Cấu hình phần cứng (bảng 3.3)
0x2C	`PERF_TOTAL`	R	0	Số chu kỳ BUSY = 1 của tác vụ
0x30	`PERF_LOAD`	R	0	Số chu kỳ DMA thực hiện LOAD_*

Bảng 3.1: Register map (tiếp theo)

Offset	Tên	Truy cập	Reset	Nội dung
0x34	PERF_COMPUTE	R	0	Số chu kỳ pha COMPUTE
0x38	PERF_STORE	R	0	Số chu kỳ DMA thực hiện STORE_C
0x3C	—	—	—	Dự trữ
0x40	DESC_PTR	R/W	0	Dành cho phần bổ sung (mục 7.2); không có thì đọc 0, ghi bị bỏ qua
0x44	ERR_DESC_ADDR	R	0	Như trên
0x48	DESC_DONE_CNT	R	0	Như trên
0x4C–0xFFC	—	—	—	Dự trữ

Bảng 3.2: Mã lỗi ERR_CODE (STATUS[7:4])

Giá trị	Ý nghĩa	Khi nào xuất hiện
0	Không lỗi	ERROR = 0
1	Cấu hình sai	Bản chụp cấu hình vi phạm REG-12; hoặc START và START_CHAIN cùng được ghi 1
2	Lỗi AXI khi đọc	LOAD_A, LOAD_B hoặc LOAD_BIAS gặp rresp $\neq$ OKAY hoặc rlast sai vị trí
3	Lỗi AXI khi ghi	STORE_C gặp bresp $\neq$ OKAY
4	Lỗi AXI khi đọc descriptor	Chỉ có với phần bổ sung (mục 7.2)
5	Tính năng không có	START_CHAIN khi phần cứng không có chế độ chuỗi
6–15	Dự trữ	Không bao giờ xuất hiện

Bảng 3.3: Bố cục HW_CONFIG

Bit	Trường	Giá trị
[7:0]	P	4 hoặc 8
[15:8]	MAX_DIM	32
[16]	HAS_CHAIN	1 nếu có chế độ chuỗi descriptor (mục 7.2)
[17]	HAS_PINGPONG	1 nếu có scratchpad ping-pong (mục 7.3)
[23:18]	dự trữ	0
[31:24]	SPEC_VER	0x10 (spec v1.0)

3.2 Yêu cầu thanh ghi

Bảng 3.4: Thanh ghi

ID	Yêu cầu
REG-01	Offset, tên, quyền truy cập và giá trị reset như bảng 3.1
REG-02	CTRL[0] START: ghi 1 là yêu cầu bắt đầu tác vụ; bit tự xóa, đọc luôn ra 0; ghi 0 không có tác dụng
REG-03	CTRL[1] SOFT_RESET: ghi 1 là yêu cầu soft reset (mục 4.5); tự xóa
REG-04	CTRL[2] START_CHAIN: khi HAS_CHAIN = 0, lệnh vẫn được chấp nhận như một tác vụ và kết thúc ngay với ERROR, ERR_CODE = 5, không truy cập bus
REG-05	Một lần ghi CTRL có SOFT_RESET = 1: chỉ SOFT_RESET có hiệu lực. Không có SOFT_RESET mà START = START_CHAIN = 1: tác vụ được chấp nhận và kết thúc ngay với ERR_CODE = 1
REG-06	STATUS.MODE = 0 với tác vụ START, = 1 với tác vụ START_CHAIN; giữ giá trị của tác vụ gần nhất. ERR_CODE = 0 khi ERROR = 0

Bảng 3.4: Thanh ghi (tiếp theo)

ID	Yêu cầu
REG-07	Thanh ghi R/W được ghi bất kỳ lúc nào, kể cả khi BUSY = 1. Đọc lại trả giá trị CPU đã ghi, không phải bản chụp. Giá trị mới áp dụng từ lần START được chấp nhận tiếp theo
REG-08	Thanh ghi R/W lưu đủ 32 bit, kể cả bit dự trữ, để kiểm tra được REG-12. Bit dự trữ của CTRL và STATUS: ghi bị bỏ qua, đọc ra 0. Offset dự trữ: đọc ra 0, ghi bị bỏ qua. Ghi vào thanh ghi chỉ đọc bị bỏ qua
REG-09	`rst_n` đưa mọi thanh ghi về giá trị reset. Soft reset giữ nguyên các thanh ghi R/W; xóa STATUS và `PERF_*`
REG-10	`PERF_*` về 0 ở chu kỳ START được chấp nhận, đếm trong lúc BUSY = 1, dừng khi tác vụ kết thúc, bão hòa ở 0xFFFF_FFFF
REG-11	Bản chụp cấu hình gồm `MATRIX_DIM`, ba `BASE_ADDR_*`, `POST_CTRL`, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT`; lấy ở chu kỳ START được chấp nhận và giữ ổn định tới hết tác vụ
REG-12	Cấu hình hợp lệ khi và chỉ khi: M, N, K trong miền bảng 2.2; `MATRIX_DIM[31:24]` = 0; `BASE_ADDR_IN`, `BASE_ADDR_WEIGHT`, `BASE_ADDR_OUT` có bit [1:0] = 0; `POST_CTRL[31:2]` = 0; `RELU_EN` = 1 kéo theo `POST_EN` = 1; nếu `POST_EN` = 1 thì `BIAS_ADDR[1:0]` = 0 và `QUANT_SHIFT[31:5]` = 0. Khi `POST_EN` = 0, `BIAS_ADDR`, `QUANT_M0`, `QUANT_SHIFT` bị bỏ qua

3.3 Ngữ nghĩa tác vụ

Bảng 3.5: Tác vụ

ID	Yêu cầu
TSK-01	START chỉ được chấp nhận nếu BUSY = 0 tại chu kỳ Register File thực hiện lệnh ghi CTRL. START khi BUSY = 1 bị bỏ qua, không đổi trạng thái, không báo lỗi
TSK-02	Ngay chu kỳ chấp nhận: chụp cấu hình (REG-11); xóa DONE, ERROR, `ERR_CODE`, `PERF_*`; đặt MODE. BUSY = 1 từ chu kỳ kế tiếp
TSK-03	Register File phát `start_pulse` và chụp cấu hình trước khi trả phản hồi B cho lệnh ghi CTRL; mọi lần đọc STATUS sau khi lệnh ghi hoàn tất đều thấy tác vụ mới
TSK-04	DONE = 1 chỉ khi STORE_C đã nhận phản hồi B cuối cùng là OKAY, tức toàn bộ kết quả đã nằm trong DRAM
TSK-05	ERROR = 1 khi gặp một lỗi ở bảng 3.2; sau lỗi không pha mới nào được bắt đầu; nội dung vùng đích trong DRAM không xác định
TSK-06	BUSY về 0 ở đúng cạnh clock mà DONE hoặc ERROR được đặt. Sau khi chấp nhận và cho tới tác vụ kế tiếp hoặc soft reset, mọi lần đọc STATUS thấy đúng một trong ba bit BUSY, DONE, ERROR bằng 1
TSK-07	DONE và ERROR giữ giá trị (sticky); chỉ bị xóa khi tác vụ mới được chấp nhận, khi soft reset hoặc khi `rst_n`
TSK-08	Sau DONE hoặc ERROR, firmware được phép ghi START ngay, không cần soft reset
TSK-09	Tín hiệu "xong" nội bộ (xong một lệnh DMA, `pe_done`) không hiện ra cho CPU; chỉ STATUS.DONE báo toàn bộ tác vụ xong

3.4 Quy tắc cho firmware

Bảng 3.6: Firmware

ID	Yêu cầu
FW-01	Trước lệnh ghi START, firmware thực hiện `fencerw, rw`, để mọi lệnh ghi A, B, bias vào DRAM đến trước lệnh MMIO
FW-02	Firmware chờ bằng polling cho tới khi DONE hoặc ERROR bằng 1, rồi thực hiện `fencerw, rw` trước khi đọc kết quả

Bảng 3.6: Firmware (tiếp theo)

ID	Yêu cầu
FW-03	Đường DMA được giả định nhất quán cache với CPU. Nếu hệ thống tích hợp không bảo đảm điều đó, firmware phải flush dòng cache của A, B, bias trước START và hủy dòng cache của vùng đích sau DONE
FW-04	Firmware không ghi vào vùng DRAM của tác vụ đang chạy

```
#define GEMM_BASE  0x60000000u
#define REG(off)   (*(volatile uint32_t *)(GEMM_BASE + (off)))
#define ST_DONE    (1u << 0)
#define ST_ERROR   (1u << 2)

int gemm_run(uint32_t a, uint32_t b, uint32_t c, uint32_t M, uint32_t N,
             uint32_t K, int post, int relu, uint32_t bias,
             int32_t m0, uint32_t sh) {
    REG(0x08) = (K << 16) | (N << 8) | M;     /* MATRIX_DIM   */
    REG(0x0C) = a;  REG(0x10) = b;  REG(0x14) = c;
    REG(0x18) = (post ? 1u : 0u) | (relu ? 2u : 0u);
    REG(0x1C) = bias;  REG(0x20) = (uint32_t)m0;  REG(0x24) = sh;
    asm volatile ("fence rw,rw");             /* FW-01 */
    REG(0x00) = 1u;                           /* START */
    uint32_t s;
    do { s = REG(0x04); } while (!(s & (ST_DONE | ST_ERROR)));
    asm volatile ("fence rw,rw");             /* FW-02 */
    return (s & ST_ERROR) ? (int)((s >> 4) & 0xFu) : 0;
}
```

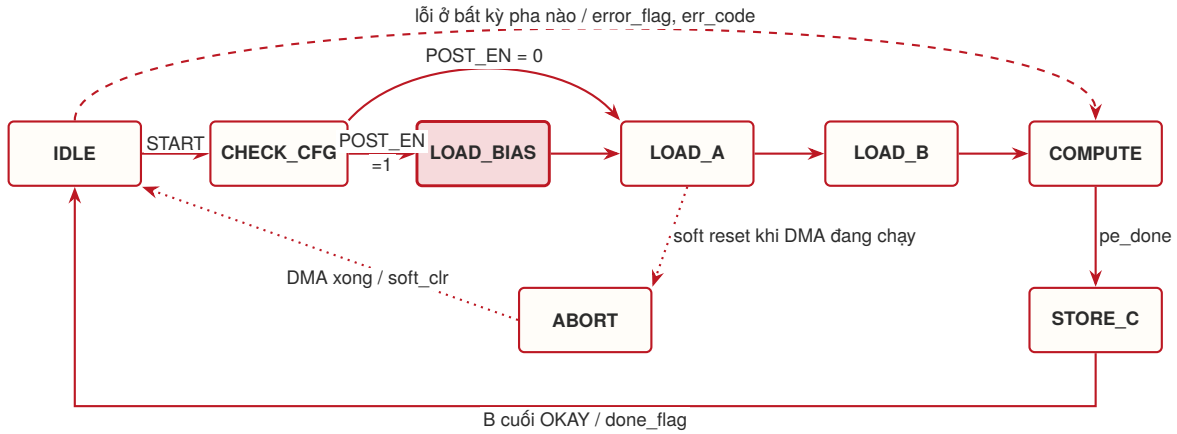
4 Luồng vận hành

Một tác vụ chạy các pha theo thứ tự CHECK_CFG → LOAD_BIAS (nếu bật hậu xử lý) → LOAD_A → LOAD_B → COMPUTE → STORE_C. Mỗi lúc chỉ một pha chạy, và mỗi pha có đúng một khối làm chính. Pha sau chỉ bắt đầu khi pha trước thành công.

4.1 Các pha của một tác vụ

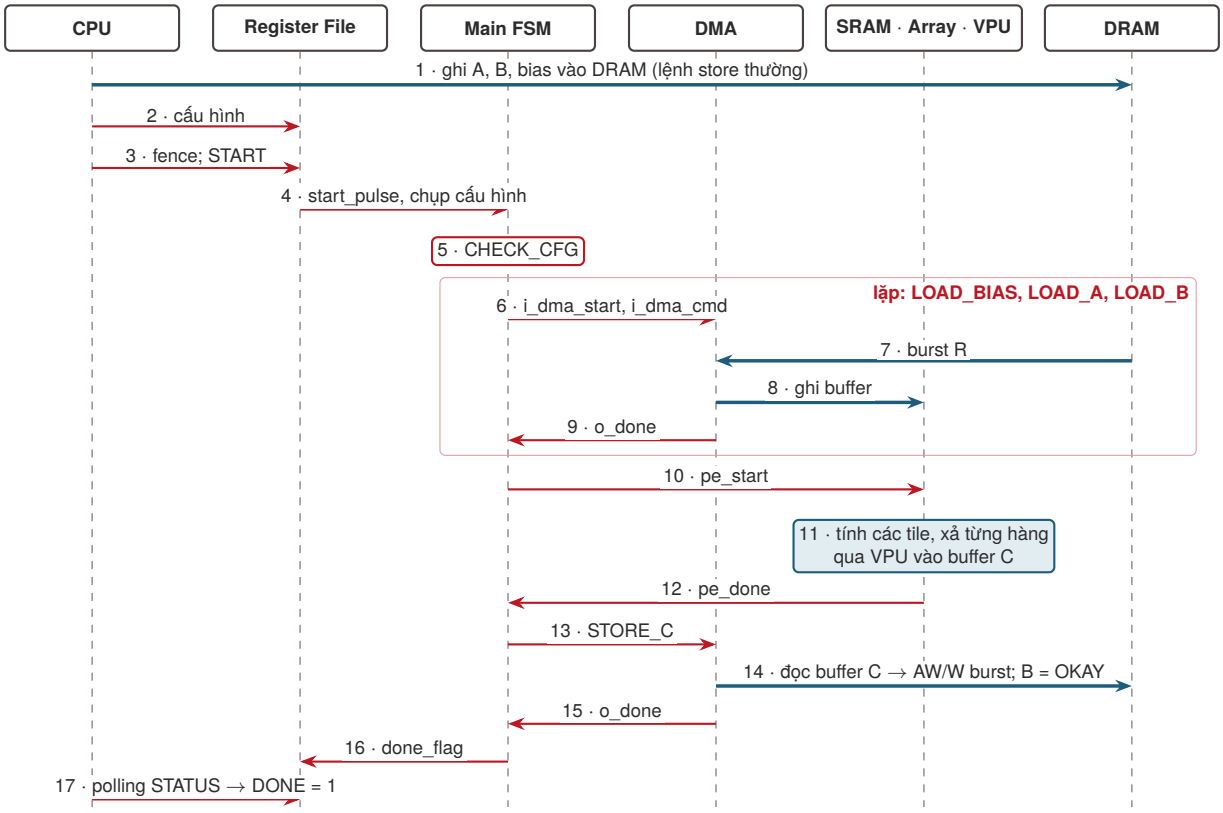
Bảng 4.1: Các pha

Pha	Khối chính	Làm gì	Xong khi
CHECK_CFG	Main FSM	Kiểm tra bản chụp cấu hình theo REG-12; sai thì báo lỗi mã 1, không truy cập bus hay scratchpad	Kiểm tra xong
LOAD_BIAS	DMA	Chép N word bias từ DRAM vào buffer BIAS. Chỉ có khi `POST_EN` = 1	SRAM nhận word cuối
LOAD_A	DMA	Chép A (M hàng × K/4 word) vào buffer A	SRAM nhận word cuối
LOAD_B	DMA	Chép B (K hàng × N/4 word) vào buffer B	SRAM nhận word cuối
COMPUTE	Array, VPU	Array tính mọi tile; mỗi hàng kết quả đi qua VPU (hậu xử lý hoặc đi thẳng) rồi ghi vào buffer C	`pe_done`
STORE_C	DMA	Chép C (M·N word) hoặc Y (M·N/4 word) từ buffer C ra DRAM	Phản hồi B cuối là OKAY



Hình 4.1: Thứ tự pha của Main FSM (minh họa). Đường lỗi và đường soft reset chỉ vẽ một mũi tên đại diện. Tên trạng thái và cách mã hóa do người hiện thực chọn; chỉ hành vi nhìn thấy từ bên ngoài là quy định

4.2 Luồng một tác vụ



Hình 4.2: Luồng một tác vụ (trường hợp thành công)

Có ba mức "xong": `o_done` của DMA báo xong một lệnh DMA; `pe_done` báo toàn bộ kết quả đã nằm trong buffer C; STATUS.DONE báo toàn bộ kết quả đã nằm trong DRAM. Chỉ mức cuối cùng hiện ra cho CPU.

4.3 Quyền truy cập scratchpad theo pha

Bảng 4.2: Ai truy cập buffer trong mỗi pha

Pha	A	B	BIAS	C
IDLE, CHECK_CFG	—	—	—	—
LOAD_BIAS	—	—	DMA ghi	—
LOAD_A	DMA ghi	—	—	—
LOAD_B	—	DMA ghi	—	—
COMPUTE	Array đọc	Array đọc	VPU đọc	VPU ghi
STORE_C	—	—	—	DMA đọc

Bảng 4.3: Quyền truy cập

ID	Yêu cầu
OWN-01	Mỗi buffer chỉ có tối đa một bên truy cập trong mỗi pha, theo bảng 4.2. Main FSM bảo đảm điều này bằng cách chạy tuần tự các pha. Vì vậy mỗi bank vật lý chỉ cần SRAM một cổng
OWN-02	Module SRAM có assertion (mô phỏng) phát hiện truy cập sai bên trong một pha

4.4 Xử lý lỗi

Bảng 4.4: Lỗi

ID	Yêu cầu
ERR-01	Cấu hình sai: CHECK_CFG kết thúc tác vụ với `ERR_CODE` = 1; không có giao dịch AXI nào, không truy cập scratchpad
ERR-02	Lỗi AXI trong một lệnh DMA: DMA ngừng phát burst mới, nhận đủ beat R hoặc phản hồi B của burst đã phát, rồi báo `o_error`. Main FSM kết thúc tác vụ với `ERR_CODE` = 2 (LOAD_*) hoặc 3 (STORE_C) và không phát lệnh nào nữa
ERR-03	Array và VPU không có đầu ra lỗi

4.5 Reset và soft reset

Bảng 4.5: Reset

ID	Yêu cầu
RST-01	`rst_n` tích cực mức thấp, bất đồng bộ. `gemm_accel_top` có bộ đồng bộ 2 flip-flop (bật bất đồng bộ, nhả đồng bộ với `clk`); đầu ra `rst_n_sync` cấp cho mọi module
RST-02	`rst_n_sync` = 0 đưa mọi trạng thái về giá trị reset. Nội dung scratchpad không bị xóa, coi là không xác định
RST-03	Soft reset không đi vào chân reset; nó là xung xóa đồng bộ `soft_clr` một chu kỳ do Main FSM phát tới Register File, DMA, SRAM, Array, VPU
RST-04	Phạm vi soft reset: FSM, DMA, Array, VPU, phần điều khiển SRAM; STATUS và `PERF_*`. Thanh ghi R/W giữ nguyên. Nội dung scratchpad không xác định
RST-05	Nếu DMA đang chạy lệnh: FSM bật `i_dma_abort`; DMA không phát burst mới, hoàn tất mọi burst đã cam kết (đã bật VALID hoặc đã gửi beat W), rồi kết thúc lệnh bằng `o_done` hoặc `o_error`. Sau đó FSM phát `soft_clr`
RST-06	Nếu DMA không chạy lệnh (IDLE, CHECK_CFG, COMPUTE): FSM phát `soft_clr` ở chu kỳ kế tiếp; COMPUTE đang chạy bị hủy
RST-07	Trong cả chuỗi soft reset: BUSY = 1; START và SOFT_RESET ghi lặp lại bị bỏ qua

Bảng 4.5: Reset (tiếp theo)

ID	Yêu cầu
RST-08	Sau `soft_clr`: STATUS = 0; tác vụ bị hủy không báo DONE cũng không báo ERROR; vùng đích trong DRAM không xác định
RST-09	Nếu yêu cầu soft reset đến đúng chu kỳ DMA kết thúc lệnh, coi như DMA đã rảnh và áp dụng RST-06. Khi đã có yêu cầu soft reset, FSM không phát lệnh mới

5 Vận hành của các khối

Mục này quy định mỗi khối làm gì và cam kết gì với các khối xung quanh: kiểu bắt tay, thông lượng, độ trễ. Port nội bộ chi tiết (tên, độ rộng, căn chu kỳ) nằm trong spec của từng module và có thể thay đổi, miễn là giữ đúng các cam kết ở đây.

Bảng 5.1: Giao tiếp nội bộ

Giữa	Kiểu	Nội dung
Register File ↔ Main FSM	Xung và mức	`start_pulse`, yêu cầu soft reset; ngược lại BUSY, xung done, xung error kèm mã, tín hiệu bận cho bộ đếm
Register File → FSM, DMA, Array, VPU	Mức, ổn định suốt tác vụ	Bản chụp cấu hình (REG-11)
Main FSM ↔ DMA	Lệnh khi DMA rảnh; done/error	Mã lệnh, abort; ngược lại bận, done, error
DMA ↔ SRAM	valid/ready	Ghi 1 word mỗi lần (A, B, BIAS); đọc 1 word mỗi lần (C)
Main FSM ↔ Array	Xung	`pe_start`, `pe_done`
SRAM → Array	Trễ cố định 1, không ready	Vector A, vector B
Array → VPU	Luồng hàng, không ready	Một hàng P giá trị INT32 kèm mask mỗi chu kỳ
VPU ↔ SRAM	Trễ cố định 1 / ghi có mask	Đọc vector bias; ghi một đoạn hàng của C
Main FSM → mọi module	Xung	`soft_clr`

5.1 Register File

CPU ↔ Register File ↔ Main FSM

TRAO ĐỔI GÌ?
CPU ghi cấu hình và lệnh, đọc trạng thái và bộ đếm. MMIO không chở dữ liệu ma trận.

AI KHỞI TẠO?
CPU là master trên AXI4-Lite; Register File chỉ trả lời. Register File phát `start_pulse` và yêu cầu soft reset tới FSM; FSM báo kết quả ngược lại.

KHI NÀO ĐƯỢC CHẤP NHẬN?
Lệnh ghi có hiệu lực khi đã nhận đủ AW và W. START chỉ được nhận khi BUSY = 0.

KHI NÀO SANG BƯỚC TIẾP?
Ngay khi chấp nhận START, Register File chụp cấu hình và phát `start_pulse`, trước khi trả phản hồi B.

Bảng 5.2: AXI4-Lite slave

ID	Yêu cầu
AXL-01	Register File nhận AW và W theo thứ tự bất kỳ; lệnh ghi chỉ thực hiện khi đã có cả hai
AXL-02	Khi `s_axi_bvalid` hoặc `s_axi_rvalid` = 1, giữ nguyên tín hiệu đó và payload cho tới khi bắt tay xong
AXL-03	Tối đa một lệnh ghi và một lệnh đọc đang xử lý; điều tiết bằng tín hiệu ready
AXL-04	Luôn trả OKAY (`bresp` = `rresp` = 2'b00), kể cả với offset dự trữ
AXL-05	Chỉ giải mã bit địa chỉ [11:2]; bit [1:0] bị bỏ qua; wrapper chỉ định tuyến vùng 4 KiB tới accelerator
AXL-06	`wstrb` áp dụng theo byte cho thanh ghi R/W: byte có strobe = 0 giữ giá trị cũ. Các bit lệnh của CTRL chỉ có tác dụng khi `wstrb[0]` = 1
AXL-07	Register File giữ cờ DONE, ERROR, `ERR_CODE`: xóa khi chấp nhận START hoặc khi `soft_clr`; đặt khi nhận xung done hoặc xung error từ FSM

5.2 Main FSM

Bảng 5.3: Main FSM

ID	Yêu cầu
FSM-01	FSM là module duy nhất ra lệnh cho DMA và array. Mọi `start_pulse` từ Register File coi như đã được chấp nhận
FSM-02	Chạy các pha theo bảng 4.1: kiểm tra REG-12 ở CHECK_CFG; bỏ LOAD_BIAS khi `POST_EN` = 0; phát đúng một lệnh DMA cho mỗi pha LOAD_* và STORE_C; phát `pe_start` đúng một lần sau LOAD_B
FSM-03	FSM chỉ phát lệnh DMA khi DMA rảnh, và chỉ phát `pe_start` khi array rảnh
FSM-04	FSM cung cấp BUSY ở mức tác vụ (không lấy từ trạng thái bận của DMA). Xung done và xung error không bao giờ cùng chu kỳ; mỗi tác vụ có tối đa một trong hai
FSM-05	Mã lỗi suy ra từ pha đang chạy: CHECK_CFG → 1; LOAD_* → 2; STORE_C → 3
FSM-06	FSM điều phối soft reset theo RST-05 … RST-09

5.3 DMA engine

Main FSM ↔ DMA ↔ DRAM / SRAM

TRAO ĐỔI GÌ?
FSM gửi một lệnh: LOAD_A, LOAD_B, LOAD_BIAS hoặc STORE_C. DMA tự tính địa chỉ và số word từ bản chụp cấu hình, rồi báo done hoặc error.

AI KHỞI TẠO?
FSM khởi tạo lệnh. Sau đó DMA là master: phát burst trên AXI4-Full và gửi yêu cầu đọc/ghi tới SRAM (valid/ready).

KHI NÀO ĐƯỢC CHẤP NHẬN?
FSM chỉ ra lệnh khi DMA rảnh. Mỗi lệnh kết thúc bằng đúng một done hoặc một error.

KHI NÀO SANG BƯỚC TIẾP?
LOAD xong khi SRAM đã nhận word cuối. STORE_C xong khi phản hồi B cuối là OKAY.

Bảng 5.4: Các lệnh DMA

Lệnh	Nguồn → đích	Số word	Địa chỉ DRAM của word thứ t
LOAD_A	DRAM → buffer A	M·K/4	`BASE_ADDR_IN` + 4t
LOAD_B	DRAM → buffer B	K·N/4	`BASE_ADDR_WEIGHT` + 4t
LOAD_BIAS	DRAM → buffer BIAS	N	`BIAS_ADDR` + 4t
STORE_C	buffer C → DRAM	M·N (INT32) hoặc M·N/4 (INT8)	`BASE_ADDR_OUT` + 4t

Bảng 5.5: DMA và AXI4-Full

ID	Yêu cầu
DMA-01	Mỗi lệnh kết thúc bằng đúng một xung done hoặc error, không bao giờ cả hai; không có done/error khi không có lệnh. Lệnh có mã không dùng → error ngay, không truy cập bus hay SRAM
DMA-02	Địa chỉ DRAM tăng liên tục theo bảng 5.4; địa chỉ scratchpad theo SPM-02. DMA chép nguyên word, không đảo byte
DMA-03	Cắt burst: tại địa chỉ addr, số beat = min(số word còn lại, `MAX_BURST_BEATS`, (4096 − addr mod 4096)/4). Burst không bao giờ vượt biên 4 KiB
DMA-04	`arsize` = `awsize` = 3'b010; `arburst` = `awburst` = 2'b01 (INCR); `arlen`/`awlen` = số beat − 1; `wstrb` = 4'b1111; `wlast` = 1 đúng ở beat cuối mỗi burst
DMA-05	VALID không phụ thuộc READY; khi VALID = 1, giữ VALID và payload tới khi bắt tay. `wvalid` không chờ `awready`
DMA-06	Tối đa 1 burst đọc outstanding (từ bắt tay AR tới beat có `rlast`) và 1 burst ghi outstanding (từ AW hoặc beat W đầu tiên tới bắt tay B)
DMA-07	DMA được phép hạ `rready` để tạo backpressure và được phép giữ `bready` = 1 liên tục. DMA nhận đủ mọi beat R và phản hồi B của mọi burst đã phát, kể cả khi lỗi hoặc abort
DMA-08	Chỉ OKAY là thành công. DMA tự đếm beat; `rlast` phải ở beat thứ `arlen` + 1, lệch thì là lỗi đọc. `rlast` đến sớm: burst coi như kết thúc; chưa có `rlast` ở beat cuối kỳ vọng: nhận tiếp tới `rlast` và bỏ beat thừa
DMA-09	Khi lỗi: ngừng phát burst mới, hoàn tất burst đang dở, rồi báo error. Dữ liệu của burst đọc bị lỗi có được ghi vào SRAM hay không là không quy định
DMA-10	Abort: không phát AR/AW mới, hoàn tất mọi burst đã bật VALID hoặc đã gửi beat W, rồi kết thúc lệnh bằng done hoặc error như thường

5.4 Local SRAM

Bảng 5.6: Scratchpad

ID	Yêu cầu
SPM-01	Bốn buffer logic tách biệt: A (256 word), B (256 word), C (1024 word), BIAS (32 word)
SPM-02	Địa chỉ word, có đệm 2D: A {i[4:0], kw[2:0]}, kw = k » 2, byte lane = k[1:0]; B {k[4:0], jw[2:0]}, jw = j » 2; C dạng INT32 {i[4:0], j[4:0]}; C dạng INT8 {i[4:0], jw[2:0]}, byte lane = j[1:0]; BIAS {j[4:0]}. Bố cục này không phải bản sao tuyến tính của DRAM; DMA là nơi chuyển đổi
SPM-03	Vị trí ngoài vùng hợp lệ của tác vụ không bị ghi và có nội dung không xác định; array chỉ được đọc chúng cho các lane không hợp lệ
SPM-04	Phía array và VPU, trong cùng một chu kỳ, scratchpad phục vụ đồng thời: 1 lần đọc A (P byte từ P hàng khác nhau), 1 lần đọc B (P byte liền nhau), 1 lần đọc BIAS (P word liền nhau), 1 lần ghi C (một đoạn hàng P phần tử, có mask). Độ trễ đọc cố định 1 chu kỳ; không có backpressure. Cách chia bank vật lý để đạt yêu cầu này do spec module SRAM quy định
SPM-05	Phía DMA: nhận 1 word ghi mỗi chu kỳ và trả 1 word đọc mỗi chu kỳ, bắt tay valid/ready; mỗi yêu cầu đọc có đúng một dữ liệu trả về, theo thứ tự
SPM-06	`rst_n` và soft reset không khởi tạo nội dung scratchpad

5.5 Systolic array

SRAM ↔ Systolic array ↔ VPU

TRAO ĐỔI GÌ?

Mỗi chu kỳ array đọc một cột của khối A và một hàng của khối B. Khi một tile xong, array xả kết quả theo hàng, mỗi chu kỳ một hàng P giá trị INT32 kèm mask, sang VPU.

AI KHỞI TẠO?

Array là master phía SRAM, tự lập lịch đọc. FSM chỉ phát `pe_start` một lần cho mỗi tác vụ.

KHI NÀO ĐƯỢC CHẤP NHẬN?

Không có ready: SRAM phải phục vụ mọi yêu cầu, VPU phải nhận mọi hàng. Dữ liệu đọc về đúng 1 chu kỳ sau yêu cầu.

KHI NÀO SANG BƯỚC TIẾP?

Array phát `pe_done` sau khi hàng cuối của tile cuối, đi qua VPU, đã được ghi vào buffer C.

Bảng 5.7: Systolic array

ID	Yêu cầu
ARR-01	Array có P×P PE, P ∈ {4, 8}. Mỗi PE gồm bộ nhân INT8 × INT8, thanh ghi tích lũy INT32 `acc` và thanh ghi kết quả INT32 `res`
ARR-02	Output-Stationary: trong tile (i0, j0), PE(r, c) tích lũy C[i0+r][j0+c]; A đi theo hàng, B đi theo cột. Với mọi k, PE(r, c) nhân đúng A[i0+r][k] với B[k][j0+c]. Làm lệch (lane r của A trễ r chu kỳ, lane c của B trễ c chu kỳ) nằm trong array
ARR-03	Mỗi tile duyệt toàn bộ K trong một lượt; không có partial sum rời PE. Phần tử k = 0 mang cờ `first`: PE nạp tích thay vì cộng dồn, nên không cần chu kỳ xóa
ARR-04	Phần tử k = K−1 mang cờ `last`: PE chép giá trị tích lũy cuối vào `res`. `res` giữ nguyên tới lần `last` kế tiếp, nên tile sau được tính trong lúc tile trước đang xả
ARR-05	Bộ tuần tự tile nằm trong array: lưới ⌈M/P⌉ × ⌈N/P⌉ tile, thứ tự duyệt tự do, mỗi tile tính đúng một lần. Tile kế tiếp bắt đầu sau max(K, P) chu kỳ
ARR-06	Xả: từ các thanh ghi `res`, mỗi chu kỳ một hàng của tile (P giá trị INT32 + mask P bit) sang VPU
ARR-07	Tile biên: hàng r hợp lệ khi i0 + r < M, cột c hợp lệ khi j0 + c < N. Không xả hàng không hợp lệ; mask = 0 cho cột không hợp lệ. Mỗi C[i][j] hợp lệ được ghi đúng một lần; không vị trí nào khác bị ghi
ARR-08	Từ `pe_start` đến `pe_done` array không bao giờ phải chờ; `pe_done` là xung 1 chu kỳ, mỗi `pe_start` có đúng một `pe_done`. Sau `soft_clr`, array về rảnh ngay và không phát `pe_done` cho lượt bị hủy

NGUYÊN LÝ PE

Mỗi chu kỳ, với cặp (a, b) hợp lệ: $sum = (first\ ?\ 0 : acc) + a \cdot b$; $acc \leftarrow sum$; nếu $last$ thì $res \leftarrow sum$. Giá trị a cùng cờ hợp lệ, $first$, $last$ đi sang PE bên phải; b đi xuống PE bên dưới; mỗi bước trễ 1 chu kỳ. Khoảng cách tile max(K, P) bảo đảm `res` của một hàng đã được xả trước khi tile kế tiếp ghi đè.

5.6 VPU

Bảng 5.8: VPU

ID	Yêu cầu
VPU-01	VPU nằm giữa đường xả của array và cổng ghi buffer C. Nó có P lane, nhận một hàng mỗi chu kỳ, không có backpressure và có độ trễ cố định `L_VPU` chu kỳ (hằng số của thiết kế)
VPU-02	`POST_EN` = 0: đầu ra bằng đầu vào (INT32), ghi vào C theo bố cục INT32
VPU-03	`POST_EN` = 1: lane c của hàng i tính FUN-03 với bias[j0 + c], M_0, sh, `RELU_EN` của bản chụp cấu hình; ghi Y vào C theo bố cục INT8, mask theo byte
VPU-04	VPU đọc bias từ buffer BIAS, P word mỗi chu kỳ, cùng nhịp với hàng đang xử lý
VPU-05	VPU không truy cập DRAM và không giữ trạng thái giữa các tác vụ

6 Port top-level

Port của `gemm_accel_top` **đã khóa**: clock, reset, một cổng AXI4-Lite slave (17 tín hiệu) và một cổng AXI4-Full master (25 tín hiệu), tổng 44 port. Mọi thay đổi về sau, kể cả phần bổ sung ở mục 7, chỉ được thay đổi port nội bộ giữa các module, không được thêm, bớt hay đổi port ở bảng 6.2 và bảng 6.3.

6.1 Tham số

Bảng 6.1: Tham số của `gemm_accel_top`

Tham số	Mặc định	Ý nghĩa
`PE_ARRAY_SIZE`	8	P; chỉ nhận 4 hoặc 8
`MAX_BURST_BEATS`	16	Số beat tối đa mỗi burst, 1 … 256
`HAS_CHAIN`	0	1 nếu hiện thực chế độ chuỗi descriptor (mục 7.2)
`HAS_PINGPONG`	0	1 nếu hiện thực scratchpad ping-pong (mục 7.3); đòi hỏi `HAS_CHAIN` = 1

6.2 Danh sách port

Bảng 6.2: Port clock, reset và AXI4-Lite slave (MMIO)

#	Port	Rộng	Chiều	Mô tả
1	`clk`	1	In	Clock duy nhất của accelerator
2	`rst_n`	1	In	Reset tích cực mức thấp, bất đồng bộ; được đồng bộ bên trong (RST-01)
3	`s_axi_awaddr`	32	In	AW: địa chỉ ghi; chỉ giải mã bit [11:2] (AXL-05)
4	`s_axi_awvalid`	1	In	AW: địa chỉ hợp lệ
5	`s_axi_awready`	1	Out	AW: Register File nhận địa chỉ
6	`s_axi_wdata`	32	In	W: dữ liệu ghi
7	`s_axi_wstrb`	4	In	W: byte strobe (AXL-06)
8	`s_axi_wvalid`	1	In	W: dữ liệu hợp lệ
9	`s_axi_wready`	1	Out	W: Register File nhận dữ liệu
10	`s_axi_bresp`	2	Out	B: luôn 2'b00 OKAY (AXL-04)
11	`s_axi_bvalid`	1	Out	B: phản hồi hợp lệ
12	`s_axi_bready`	1	In	B: CPU nhận phản hồi
13	`s_axi_araddr`	32	In	AR: địa chỉ đọc; chỉ giải mã bit [11:2]
14	`s_axi_arvalid`	1	In	AR: địa chỉ hợp lệ
15	`s_axi_arready`	1	Out	AR: Register File nhận địa chỉ
16	`s_axi_rdata`	32	Out	R: dữ liệu đọc
17	`s_axi_rresp`	2	Out	R: luôn 2'b00 OKAY
18	`s_axi_rvalid`	1	Out	R: dữ liệu hợp lệ
19	`s_axi_rready`	1	In	R: CPU nhận dữ liệu

Bảng 6.3: Port AXI4-Full master (DMA)

#	Port	Rộng	Chiều	Mô tả
20	`m_axi_araddr`	32	Out	AR: địa chỉ byte đầu burst đọc
21	`m_axi_arlen`	8	Out	AR: số beat − 1
22	`m_axi_arsize`	3	Out	AR: luôn 3'b010

Bảng 6.3: Port AXI4-Full master (DMA) (tiếp theo)

#	Port	Rộng	Chiều	Mô tả
23	`m_axi_arburst`	2	Out	AR: luôn 2'b01 (INCR)
24	`m_axi_arvalid`	1	Out	AR: yêu cầu hợp lệ
25	`m_axi_arready`	1	In	AR: bus nhận yêu cầu
26	`m_axi_rdata`	32	In	R: dữ liệu một beat
27	`m_axi_rresp`	2	In	R: mã phản hồi
28	`m_axi_rlast`	1	In	R: beat cuối của burst
29	`m_axi_rvalid`	1	In	R: dữ liệu hợp lệ
30	`m_axi_rready`	1	Out	R: DMA nhận dữ liệu
31	`m_axi_awaddr`	32	Out	AW: địa chỉ byte đầu burst ghi
32	`m_axi_awlen`	8	Out	AW: số beat − 1
33	`m_axi_awsize`	3	Out	AW: luôn 3'b010
34	`m_axi_awburst`	2	Out	AW: luôn 2'b01 (INCR)
35	`m_axi_awvalid`	1	Out	AW: yêu cầu hợp lệ
36	`m_axi_awready`	1	In	AW: bus nhận yêu cầu
37	`m_axi_wdata`	32	Out	W: dữ liệu một beat
38	`m_axi_wstrb`	4	Out	W: luôn 4'b1111
39	`m_axi_wlast`	1	Out	W: beat cuối của burst
40	`m_axi_wvalid`	1	Out	W: dữ liệu hợp lệ
41	`m_axi_wready`	1	In	W: bus nhận dữ liệu
42	`m_axi_bresp`	2	In	B: mã phản hồi ghi
43	`m_axi_bvalid`	1	In	B: phản hồi hợp lệ
44	`m_axi_bready`	1	Out	B: DMA nhận phản hồi

Bảng 6.4: Port top-level

ID	Yêu cầu
TOP-01	Port của `gemm_accel_top` đúng tên, độ rộng và chiều như bảng 6.2 và bảng 6.3; không có port nào khác, kể cả ngắt
TOP-02	Danh sách port này cố định cho mọi giá trị tham số ở bảng 6.1
TOP-03	Giá trị tham số ngoài miền cho phép phải bị chặn lúc elaboration
TOP-04	Kênh đọc AXI (AR, R) và kênh ghi (AW, W, B) do DMA điều khiển; không có đường tổ hợp từ input tới output trên cùng một cổng AXI
TOP-05	`gemm_accel_top` là Chisel BlackBox, nối hệ thống qua cầu TLToAXI4 (MMIO) và AXI4ToTL (DMA). Tín hiệu AXI phụ (ID, LOCK, CACHE, PROT, QOS) do `GemmWrapper.scala` cấp

7 Thành phần bổ sung

Hai thành phần dưới đây chỉ làm nếu còn thời gian: **DMA theo chuỗi descriptor** (CPU ra lệnh một lần cho nhiều phép GEMM) và **scratchpad ping-pong** (nạp, tính, ghi của các phép GEMM kế nhau chạy chồng lên nhau). Chúng đã được đặc tả đầy đủ ở đây và dùng các offset, bit đã dành sẵn, nên khi hiện thực không phải sửa các mục 1 – 6.

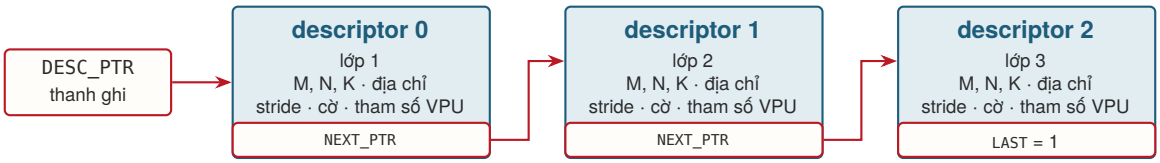
7.1 Nguyên tắc chung

Bảng 7.1: Nguyên tắc của phần bổ sung

ID	Yêu cầu
BS-01	Không thêm, bớt hay đổi port top-level (mục 6); không đổi offset hay bit đã định nghĩa ở bảng 3.1. Chỉ dùng CTRL[2], STATUS[3], các offset 0x40 … 0x48 và bit `HW_CONFIG[17:16]`
BS-02	Bật bằng tham số `HAS_CHAIN`, `HAS_PINGPONG`; phần cứng báo qua `HW_CONFIG`. Firmware phải đọc `HW_CONFIG` trước khi dùng phần bổ sung
BS-03	Tác vụ START (chế độ đơn) giữ nguyên mọi hành vi của mục 1 – 6, dù phần bổ sung có hay không
BS-04	Phần bổ sung chỉ thay đổi port nội bộ: DMA tách thành kênh đọc và kênh ghi chạy độc lập, thêm lệnh FETCH; các cổng SRAM thêm bit chọn bank; FSM thêm bộ lập lịch và hàng đợi descriptor

7.2 DMA theo chuỗi descriptor

CPU dựng sẵn một danh sách descriptor trong DRAM, mỗi descriptor mô tả một phép GEMM. CPU ghi địa chỉ descriptor đầu vào `DESC_PTR`, thực hiện `fencerw,rw`, ghi CTRL[2] = 1 (START_CHAIN), rồi chờ DONE một lần cho cả chuỗi. Accelerator tự đọc từng descriptor và chạy hết chuỗi.



Hình 7.1: Chuỗi descriptor: một lệnh START_CHAIN chạy ba phép GEMM liên tiếp

Bảng 7.2: Bố cục descriptor (16 word, 64 byte)

Offset	Word	Tên	Bit	Nội dung
0x00	0	`DIM`	[31:0]	M, N, K, bố cục như `MATRIX_DIM`
0x04	1	`A_ADDR`	[31:0]	Địa chỉ byte của A[0][0]
0x08	2	`B_ADDR`	[31:0]	Địa chỉ byte của B[0][0]; bỏ qua khi `REUSE_B` = 1
0x0C	3	`C_ADDR`	[31:0]	Địa chỉ byte của C[0][0] hoặc Y[0][0]
0x10	4	`LDA, LDB`	[15:0], [31:16]	Stride theo hàng của A và B (byte); 0 = các hàng liền nhau
0x14	5	`LDC`	[15:0]	Stride của C hoặc Y (byte); 0 = liền nhau; [31:16] dự trữ
0x18	6	`FLAGS`	[4:0]	[0] `LAST`, [1] `WAIT_PREV`, [2] `REUSE_B`, [3] `POST_EN`, [4] `RELU_EN`; [31:5] dự trữ
0x1C	7	`NEXT_PTR`	[31:0]	Địa chỉ descriptor kế tiếp, căn 64 byte; bỏ qua khi `LAST` = 1
0x20	8	`BIAS_ADDR`	[31:0]	Như thanh ghi `BIAS_ADDR`; chỉ dùng khi `POST_EN` = 1
0x24	9	`QM0`	[31:0]	Như `QUANT_M0`; chỉ dùng khi `POST_EN` = 1
0x28	10	`QSHIFT`	[4:0]	Như `QUANT_SHIFT`; [31:5] dự trữ
0x2C	11	`TAG`	[31:0]	Tùy phần mềm; phần cứng không đọc
0x30–0x3C	12–15	—	—	Dự trữ, phải bằng 0

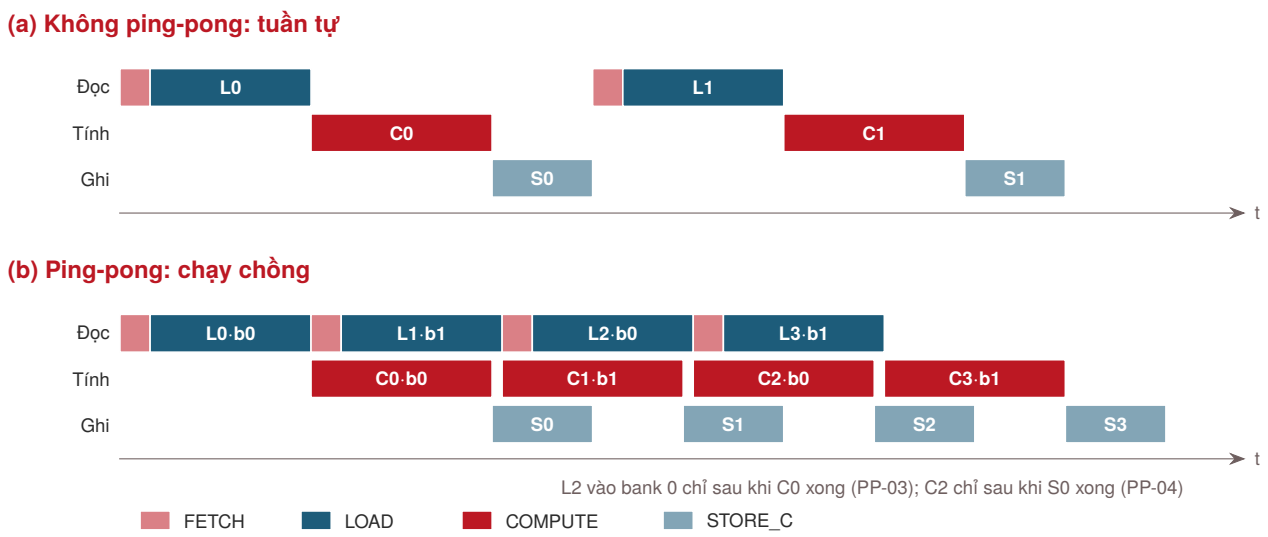
Với stride, địa chỉ phần tử là: A[i][k] tại `A_ADDR` + i·`LDA` + k; B[k][j] tại `B_ADDR` + k·`LDB` + j; C[i][j] tại `C_ADDR` + i·`LDC` + 4j; Y[i][j] tại `C_ADDR` + i·`LDC` + j. Stride = 0 được hiểu là đúng độ dài hàng.

Bảng 7.3: Chế độ chuỗi

ID	Yêu cầu
DSC-01	DESC_PTR và mọi NEXT_PTR được dùng phải căn 64 byte; nếu không, ERR_CODE = 1. Mỗi descriptor được đọc đúng một lần, theo thứ tự chuỗi, bằng một burst INCR 16 beat (lệnh DMA FETCH)
DSC-02	Mỗi descriptor chạy các pha FETCH → kiểm tra → LOAD_BIAS (nếu POST_EN) → LOAD_A → LOAD_B (trừ khi REUSE_B) → COMPUTE → STORE_C, với nội dung giống hệt một tác vụ chế độ đơn có cùng tham số
DSC-03	Descriptor hợp lệ khi: M, N, K đúng miền; DIM[31:24] = 0; mọi địa chỉ được dùng căn 4 byte; mỗi stride bằng 0 hoặc ≥ độ dài hàng (K cho A; N cho B; 4N cho C; N cho Y) và là bội của 4; bit và word dự trữ bằng 0; RELU_EN kéo theo POST_EN; REUSE_B chỉ khi không phải descriptor đầu và K, N bằng của descriptor trước. Sai → ERR_CODE = 1
DSC-04	REUSE_B = 1: không nạp B; COMPUTE dùng B của descriptor ngay trước
DSC-05	WAIT_PREV = 1: các pha LOAD của descriptor chỉ bắt đầu khi STORE_C của mọi descriptor trước nó đã xong. Phần mềm phải đặt cờ này khi descriptor đọc dữ liệu do descriptor trước ghi ra
DSC-06	Chuỗi kết thúc ở descriptor đầu tiên có LAST = 1. DONE = 1 khi STORE_C của descriptor đó nhận phản hồi B cuối là OKAY
DSC-07	DESC_DONE_CNT tăng 1 mỗi khi STORE_C của một descriptor xong OKAY; các descriptor xong đúng theo thứ tự chuỗi
DSC-08	Lỗi: gọi f là descriptor có thứ tự nhỏ nhất bị lỗi. Không pha nào của descriptor thứ ≥ f được bắt đầu; các descriptor đứng trước f được chạy xong. Khi mọi khối đã rảnh: ERROR = 1, ERR_CODE theo bảng 3.2 (lỗi khi FETCH là mã 4), ERR_DESC_ADDR = địa chỉ của f, DESC_DONE_CNT = số thứ tự của f
DSC-09	Khi dùng stride lớn hơn độ dài hàng, DMA cắt burst theo từng hàng: burst không bao giờ trải qua khoảng hở giữa hai hàng, và các byte trong khoảng hở không bị đọc hay ghi
DSC-10	Accelerator không ghi vào descriptor; phần mềm không sửa descriptor và vùng dữ liệu của chuỗi đang chạy. Vùng đích của mọi descriptor không được chồng lên vùng descriptor
DSC-11	Ở chế độ chuỗi, tham số VPU lấy từ descriptor; các thanh ghi MATRIX_DIM, BASE_ADDR_*, POST_CTRL, BIAS_ADDR, QUANT_* không được dùng

7.3 Scratchpad ping-pong

Mỗi buffer A, B, C, BIAS có hai bank. DMA kênh đọc nạp descriptor d+1 vào một bank trong lúc array tính descriptor d trên bank kia, và DMA kênh ghi chép kết quả của descriptor d−1 ra DRAM. Thời gian mỗi descriptor tiến tới max(nạp, tính, ghi) thay vì tổng của chúng. Phần này đòi hỏi chế độ chuỗi.



Hình 7.2: Lịch chạy minh họa với thời lượng giả định (đơn vị tùy ý). "·b0": bank được dùng

Bảng 7.4: Chọn bank cho descriptor d

Buffer	Quy tắc
A, C	Đảo bank sau mỗi descriptor
B	Đảo sau mỗi lần có LOAD_B; với `REUSE_B` = 1, dùng lại bank của descriptor trước
BIAS	Đảo sau mỗi descriptor có `POST_EN` = 1

Bảng 7.5: Ping-pong

ID	Yêu cầu
PP-01	Ba khối thực thi: Đọc (FETCH, kiểm tra, LOAD_*; kênh đọc DMA), Tính (COMPUTE; array và VPU), Ghi (STORE_C; kênh ghi DMA). Mỗi khối làm một việc một lúc, theo thứ tự descriptor. Kênh đọc và kênh ghi được phép có burst AXI cùng lúc, mỗi kênh tối đa 1 burst outstanding
PP-02	Khối Đọc làm lần lượt FETCH(d), kiểm tra d, các LOAD của d, rồi mới sang FETCH(d+1)
PP-03	LOAD vào một bank chỉ bắt đầu khi COMPUTE cuối cùng đọc bank đó đã xong
PP-04	COMPUTE(d) bắt đầu khi mọi LOAD của d và COMPUTE(d−1) đã xong, và STORE_C cuối cùng đọc bank C của d đã xong
PP-05	STORE_C(d) bắt đầu khi COMPUTE(d) và STORE_C(d−1) đã xong; do đó kết quả ra DRAM đúng theo thứ tự descriptor
PP-06	`WAIT_PREV` (DSC-05) chặn các LOAD, không chặn FETCH và kiểm tra
PP-07	Tại mỗi thời điểm, mỗi bank có tối đa một bên truy cập. Hai bank khác nhau của cùng buffer được truy cập độc lập trong cùng chu kỳ; mỗi bank vẫn chỉ cần SRAM một cổng
PP-08	PP-03 … PP-06 là điều kiện cần; phần cứng được phép bắt đầu một pha muộn hơn, không bao giờ sớm hơn. Nội dung DRAM sau tác vụ phải giống hệt khi chạy không ping-pong
PP-09	Soft reset: abort cả hai kênh DMA đang bận; `soft_clr` phát khi cả hai kênh đã rảnh
PP-10	`PERF_LOAD` + `PERF_COMPUTE` + `PERF_STORE` được phép lớn hơn `PERF_TOTAL`; phần chênh là thời gian chạy chồng